

Retrieval Augmented Generation (RAG)



Document
Intelligence





Why a normal LLM is not enough?

Knowledge Cutoff

Models know the public internet up to a cutoff date. but know NOTHING about your private documents, last week's data, or internal knowledge.

e.g. GPT-4 trained until Apr 2024. Ask about last week? Hallucination city.

Private Data

Internal PDFs, company SOPs, assignment briefs, and policies are not in model training data.

e.g. Company's HR policy, product docs, internal Q&A, the LLM has never seen it.

Confident Hallucination

LLMs answer with authority even when wrong. The model doesn't know what it doesn't know.

For document QA, users must be able to verify the answer from source pages.

Retraining Cost

Fine-tuning Models on your data: expensive, slow, and gets stale again in months.



Important distinction

RAG does not retrain the LLM. It adds a document lookup step before generation.





Why RAG?

LLM memory
(training patterns)

Current prompt
(what you paste now)

External documents
(retrieved at runtime)

Three Ways to Give LLMs Fresh Knowledge:

Fine-Tuning

Re-train the model on new data.

- ✓ Model deeply learns domain
- ✗ Expensive (\$\$\$)
- ✗ Gets stale quickly
- ✗ Doesn't update in real-time

Large Context Window

Stuff everything into the prompt.

- ✓ Simple, no infrastructure
- ✓ No retrieval errors
- ✗ Expensive per query
- ✗ Struggles >100k tokens
- ✗ Slow with huge docs

RAG ★

Retrieve only relevant chunks, augment the prompt.

- ✓ Always fresh
- ✓ Cost-efficient
- ✓ Citable sources
- ✓ Works on private data
- ✗ Retrieval can fail

- Training data is not a live connection to your files.
- Pasting the full PDF can exceed context limits and add noise.
- RAG retrieves only the useful passages and places them into the prompt.



RAG

The core idea

RAG (Retrieval-Augmented Generation) is a technique that enhances LLM responses by retrieving relevant documents from an external knowledge base and injecting them into the prompt before generation.

Think of RAG as an Open-Book Exam

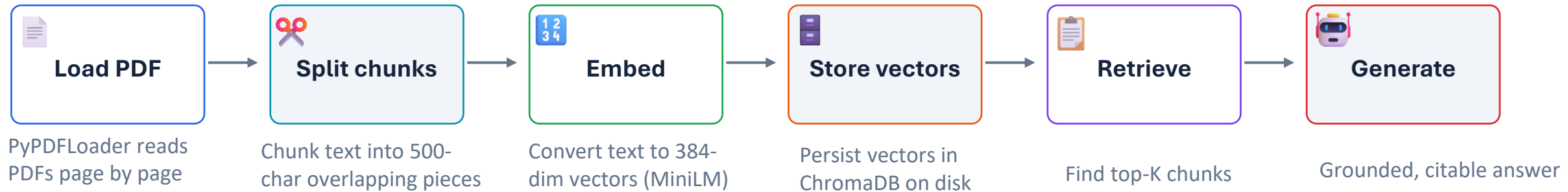
Closed-book exam = Standard LLM
Only uses what it memorized in training

Open-book exam = RAG
Retrieves relevant pages, reads them, then answers

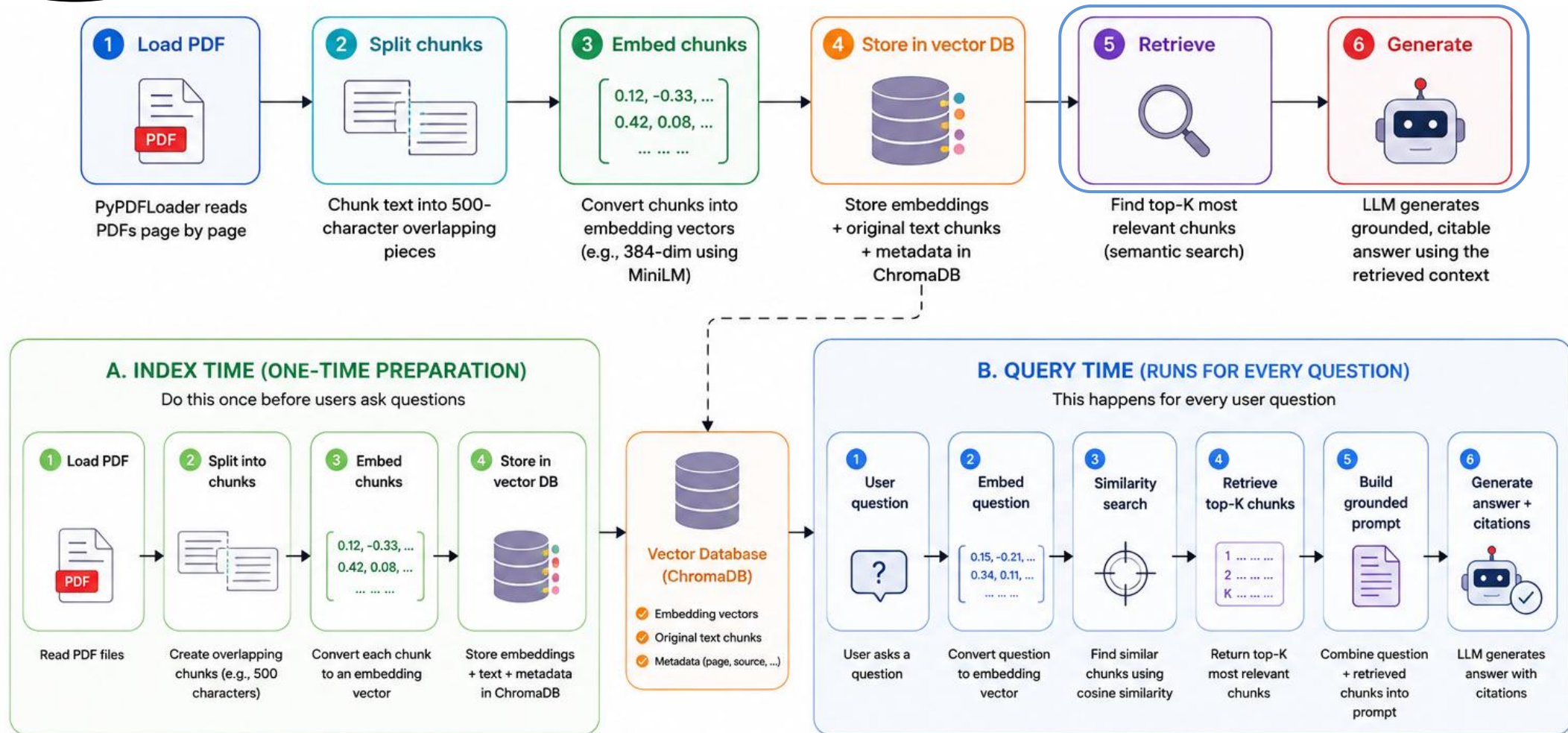
The model doesn't need to memorize everything, it knows HOW to read and synthesize.

[Lewis et al., 2020 - Facebook AI Research]





- **Indexing** happens before asking questions: load, split, embed, store.
- **Query time** happens for each question: embed question, retrieve, generate.
- **Evaluation** checks both retrieval quality and final answer quality.



Chunking & Embedding: The Critical Steps

Chunking Strategies

Fixed Size

Split every N tokens (e.g. 500). Simple, but can cut mid-sentence.

Recursive

Split on `\n\n` → `\n` → space. Preferred for prose. LangChain default.

Semantic

Use embeddings to find natural break points. Better quality, slower.

Document-aware

Respects Markdown headers, PDF structure, code blocks.

Embedding Models

OpenAI text-embedding-3-small

Best quality, paid API, 1536-dim

all-MiniLM-L6-v2 (HuggingFace)

Fast, free, 384-dim. Great for local.

BGE-M3

Multilingual, state-of-art open-source

Cohere embed-v3

Good multilingual support

 Use the SAME embedding model for indexing AND querying!



LangChain

LangChain is a framework that provides composable building blocks for LLM applications (document loaders, text splitters, vector stores, retrievers, chains, and agents) all with a unified interface.



Unified Interface

Switch between OpenAI, Claude, Gemini, Llama with one line change



100+ Integrations

Loaders for PDF, Word, Notion, GitHub, Slack and more



LCEL Chains

LangChain Expression Language: composable, streaming-ready pipelines



Agents & Tools

LLM decides which tools to call — search, calculator, APIs



LangSmith

Built-in tracing, debugging, and evaluation for production



LangGraph

Stateful multi-agent workflows as graphs



RAG

Chunking

Chunk size and overlap strongly affect retrieval quality.

Too small
missing context

Balanced
focused + enough
context

Too large
noisy retrieval

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
    separators=["\n\n", "\n", ". ", " ", ""]
)
```

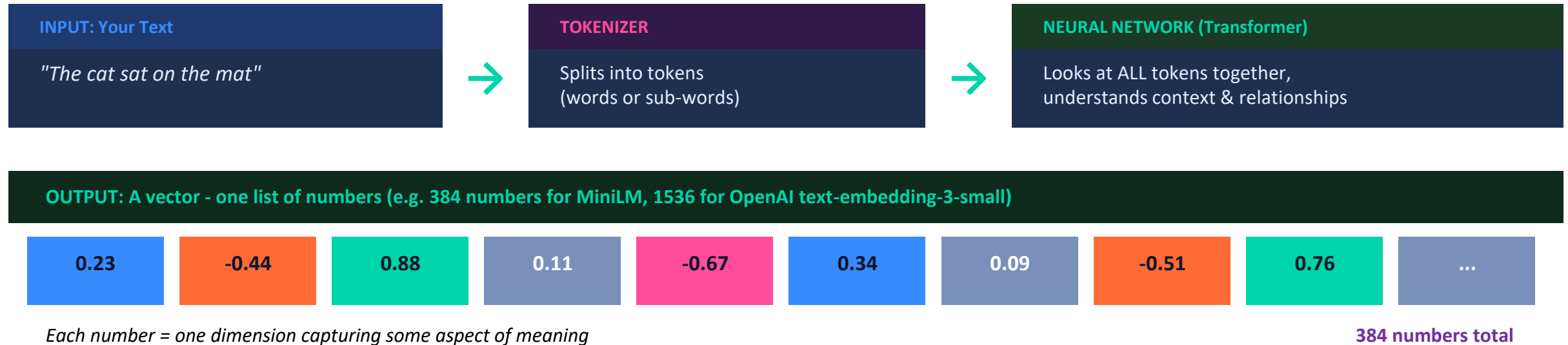
- **chunk_size** controls maximum text per chunk
- **chunk_overlap** repeats nearby context between chunks
- **separators** tell the splitter where to cut first

[practice_01_load_chunk.py](#)



What exactly is an embedding?

Text becomes a list of numbers $\Rightarrow$ Similarity becomes math



The **EMBEDDING MODEL**, a separate neural network trained specifically to convert text into vectors.

Common models: all-MiniLM-L6-v2 (free, local, 384-dim) · OpenAI text-embedding-3-small (API, 1536-dim) · BGE-M3 (multilingual)

❖ Modern embedding models (BERT, OpenAI, MiniLM) read the ENTIRE input at once and produce ONE vector that captures the overall meaning.

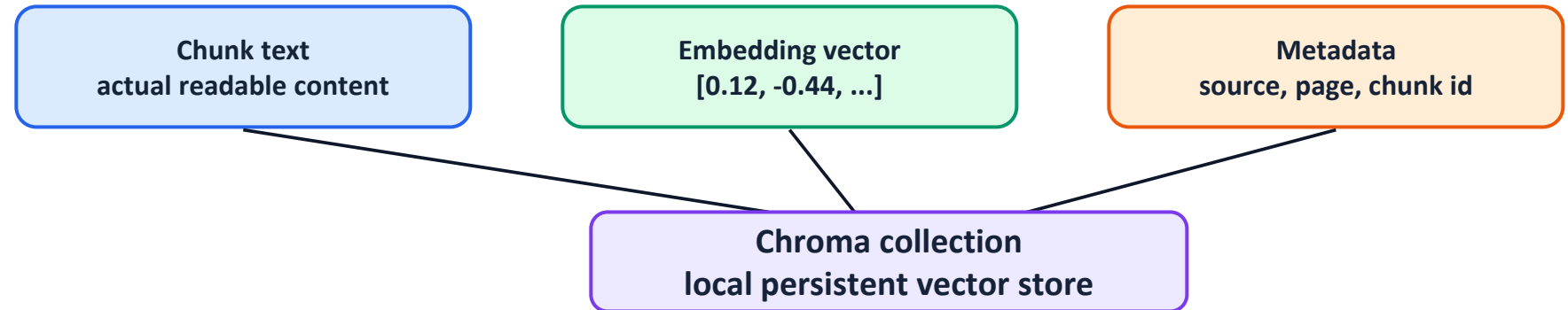
RAG

Chunk & Embedding

Embed + store: text becomes searchable numbers

A vector store keeps text, vectors, and metadata together for retrieval.

What Chroma stores



```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=CHROMA_DIR,
    collection_name="rag_demo",
)
```

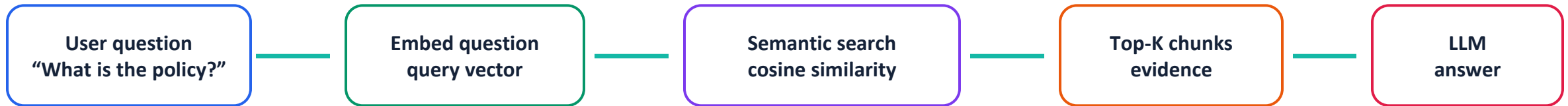
[practice_02_embed_store.py](#)



RAG

Retrieve & generate

Before the LLM writes, inspect what the retriever found. Bad retrieval usually means bad answers.



Grounded prompt = user question + retrieved chunks + answer rules

```
docs = retriever.invoke(question)

prompt = f"""Question: {question}
Retrieved context:
{format_context(docs)}
Answer using only the context."""

response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[system_msg, user_msg],
    temperature=0.2,
)
```

Debug practice: vary top-k and inspect retrieved chunks

```
for k in [1, 2, 4, 8]:
    docs = vector_store.similarity_search(question, k=k)
    print("page", page_number, doc.page_content[:350])
```

- Top-k too low → missing evidence
- Top-k too high → noisy context
- The vector DB retrieves; the LLM generates

`practice_03_retrieve_generate.py`

`practice_04_inspect_retrieval.py`



Before the LLM writes, inspect what the retriever found. Bad retrieval usually means bad answers.

```
SYSTEM: You are a document-based AI assistant.  
Use only the provided context. If missing, say you cannot find it.
```

```
USER:
```

```
Question: ...
```

```
Retrieved context:
```

```
[Source 1 | page 3] ...
```

```
[Source 2 | page 4] ...
```

```
Answer with page references.
```

- Use a refusal rule for unsupported questions.
- Ask for page/source references.
- Keep temperature low for factual Q&A.
- Do not hide retrieved evidence during testing.

`practice_05_gradio_app.py`



RAG Trade-offs

Advantages

- ✓ No retraining needed — update docs, not models
- ✓ Cost-efficient at scale — short prompts
- ✓ Citable and auditable sources
- ✓ Works on private / sensitive data
- ✓ Real-time freshness via DB updates
- ✓ Fine-grained access control per document

Challenges

- ✗ Retrieval can fail (wrong chunks, missed context)
- ✗ Chunking strategy is an art — hard to tune
- ✗ Pipeline complexity: more components = more failure points
- ✗ Embedding consistency required across model changes
- ✗ Latency overhead vs. pure LLM call
- ✗ Evaluation is hard — how do you measure retrieval quality?



Which Approach to Use?

Use RAG when...

- You have >50 documents or a large knowledge base
- Data changes frequently (news, product catalog, policies)
- You need source attribution / citations
- Per-query cost matters (production at scale)
- You need access control over documents

Use LCW when...

- Corpus fits in <100k tokens comfortably
- Task requires deep cross-document reasoning
- You're prototyping and want simplicity
- Retrieval errors would be catastrophic
- E.g. analyze one contract or one research paper

Combine both when...

- Retrieve top-k chunks with RAG (efficiency)
- Pass them to model with large context for reasoning
- Best of both worlds for complex Q&A
- Example: financial analysis across 1000s of reports



Making RAG More Effective & Reliable

Basic RAG gets you 60-70% accuracy. These techniques take you to 90%+:



Hybrid Search

BM25 + semantic



Reranking

Cross-encoder re-scores retrieved chunks



Prompt Fusion

Run multiple query rewrites in parallel



Contextual Summaries

Pre-summarize chunks for better match



Metadata Filtering

Filter by date, source, category first



Structured Extraction

Parse tables/lists before embedding



Metadata Filtering

Scope retrieval before similarity search

```
# Add metadata when loading
for doc in docs:
    doc.metadata["year"] = "2024"
    doc.metadata["dept"] = "legal"
    doc.metadata["type"] = "contract"

# Filter at query time
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={
        "k": 4,
        "filter": {
            "dept": "legal",
            "year": "2024"
        }
    }
)

# Result: searches ONLY 2024 legal docs!
```

Why metadata filtering?

Without filtering: all 50,000 chunks are searched.

With filtering: narrow to 500 relevant chunks first, then embed-search those.

Result: faster, more accurate, cheaper.

Filter by:

-  Date range (e.g., last 3 months)
-  Document type (PDF, contract, FAQ)
-  Department or source system
-  Category tag or topic
-  Document version or status



Evaluating RAG Quality



Faithfulness

Is the answer supported by the retrieved context? No hallucinations?



Answer Relevance

Does the answer actually address the user's question?



Context Recall

Did retrieval find all the chunks needed to answer?



Context Precision

Are the retrieved chunks relevant? (Low = noisy retrieval)

Free Evaluation Tools:

RAGAS

github.com/explodinggradients/ragas

TruLens

github.com/truera/trulens

DeepEval

github.com/confident-ai/deepeval



Some advance RAG Techniques

HyDE

Hypothetical Document Embeddings

Generate a fake 'ideal answer', embed it, then search. Bridges query-document embedding gap.

Query Rewriting

Multi-query retrieval

Generate 3-5 variations of the user query, retrieve for each, merge results. More recall.

Re-Ranking

Cross-encoder scoring

After top-20 retrieval, use a cross-encoder to rerank top-5. Slow but precise.

Contextual Retrieval

Prepend chunk context (Anthropic)

Ask LLM to summarize where each chunk fits in the doc. Reduces lost-in-middle issues.

RAPTOR

Recursive summarization tree

Build a tree of summaries at multiple levels. Answer from the right level.

Self-RAG

LLM decides when to retrieve

Model generates retrieval tokens to decide IF and WHAT to retrieve. More efficient.



Some advance RAG Techniques (Beyond Naive RAG)

Agentic RAG

LLM acts as an agent: decides WHEN to retrieve, WHAT to search, iterates until confident. Uses LangGraph or AutoGen.

Corrective RAG (CRAG)

After retrieval, a grading step checks if chunks are relevant. If not, triggers a web search fallback. Self-correcting.

Modular RAG

Mix-and-match pipeline components: query transform → retrieve → re-rank → fuse → generate. Highly customizable.

Web-Augmented RAG

Combines local vector DB with real-time web search (e.g. Tavily API + LangChain). Best for current events.

Structured Data RAG

SQL + NL: convert questions to SQL queries (Text2SQL). Access databases, not just documents.

RAG Fusion

Run multiple queries → retrieve → reciprocal rank fusion. Better recall than single-query RAG.



GraphRAG

What is GraphRAG?

Standard RAG stores text chunks. GraphRAG builds a KNOWLEDGE GRAPH, entities, relationships, and communities, enabling reasoning across connected facts.

Developed by Microsoft Research (2024), open-sourced as [microsoft/graphrag](https://github.com/microsoft/graphrag) on GitHub.

How it works:

1. Extract entities & relationships from documents
2. Build a graph (nodes = entities, edges = relationships)
3. Detect communities (clusters of related entities)
4. Generate community summaries
5. At query time: traverse graph, retrieve relevant subgraphs + text

✓ When GraphRAG Wins

- Complex relationship queries
- 'How are X and Y related?'
- Cross-document entity linking
- Research, legal, biomedical domains
- Multi-hop reasoning

⚠ Trade-offs

- Expensive to build (LLM extracts entities)
- Slower than vector search
- Graph DB required (Neo4j, Neptune)
- Best for complex knowledge bases
- Overkill for simple Q&A



RAG Limitations & Common Failure Cases

✗ Wrong chunks retrieved

Why: Query embedding $\neq$ answer embedding

Fix: HyDE, query rewriting, re-ranking

✗ Chunks lack context

Why: 'It was decided in 2023...' — who decided what?

Fix: Contextual retrieval, sliding window chunks

✗ Too many irrelevant chunks

Why: Top-5 chunks dilute the good answer

Fix: Cross-encoder re-ranking

✗ Multi-hop questions fail

Why: "Who founded the company that made X?" needs 2 retrievals

Fix: Agentic RAG, GraphRAG

✗ Complex relationships

Why: "How are A, B, and C related?"

Fix: GraphRAG, knowledge graphs

✗ Stale data

Why: Vector DB not updated when docs change

Fix: Automated ingestion pipelines

